Workshop 3

Pub/Sub Events Documentation

Students:

Juan David Castaneda Cardenas
Laura Vanesa Alvarez Lafont
Nicolas Andres Bolanos Fernandez
Santiago Andres Fetecua Pulgarin
Sergio Nicolas Correa Escobar

Professor:

Carlos Andres Sierra Virguez

Software Engineering II
School of Engineering
Universidad Nacional de Colombia

This document describes the Pub/Sub message formats and topics that each microservice may asynchronously consume from Kafka, the selected broker.

The topics that are created in this system are the following:

- **email.send:** Every microservice that requires to send an email publishes to this topic.

1. SignUp and LogIn

This microservice does not subscribe to any topic of the Pub/Sub server, but it publishes to email.send.

2. Event Management

This microservice does not subscribe to any topic of the Pub/Sub server, it does not publish to any either.

3. Confirmation

This microservice does not subscribe to any topic of the Pub/Sub server, but it publishes to email.send.

4. Email Notification

Every message sent to the email.send topic, which this microservice is subscribed to, must follow this JSON structure:

```JSON
{
  "to": "string",
  "subject": "string",
  "type": "EmailType",
  "params": { ... }
}
```

Field	Type	Required	Description

`to`	string	Yes	Recipient email address
`subject`	string	Yes	Email subject line
`type`	EmailType (enum)	Yes	Template to use
`params`	Map<string, object>	Yes	Template parameters (depend on the email type)

Depending on the type, the params must be different, as shown below.

a. SUCCESSFUL_REGISTER

Param	Type	Description
`name`	string	User's name

An example is presented:

```JSON
{
  "to": "user@example.com",
  "subject": "User Registered",
  "type": "SUCCESSFUL_REGISTER",
  "params": {
    "name": "John Doe"
  }
}
```

b. PASSWORD_RESET

Param	Type	Description
`name`	string	User's name
`token`	string	Password reset code

An example is presented:

```JSON
{
  "to": "user@example.com",
  "subject": "Reset your password",
  "type": "PASSWORD_RESET",
  "params": {
    "name": "John Doe",
    "token": "823912"
  }
}
```

c. EVENT_CONFIRMATION

Param	Type	Description
`name`	string	User's name
`eventName`	string	Event name
`eventDate`	string	Event date
`location`	string	Event location

An example is presented:

```JSON
{
  "to": "user@example.com",
  "subject": "You're confirmed!",
  "type": "EVENT_CONFIRMATION",
  "params": {
    "name": "John Doe",
    "eventName": "Tech Summit",
    "eventDate": "2025-05-22",
    "location": "Main Auditorium"
  }
}
```

d. WAITLIST_NOTIFICATION

Param	Type	Description
`name`	string	User's name
`eventName`	string	Event name

An example is presented:

```
JSON
{
  "to": "user@example.com",
  "subject": "You're on the waitlist",
  "type": "WAITLIST_NOTIFICATION",
  "params": {
    "name": "John Doe",
    "eventName": "Tech Summit"
  }
}
```

e. WAITLIST_PROMOTION

Param	Type	Description
`name`	string	User's name
`eventName`	string	Event name
`eventDate`	string	Event date
`location`	string	Event location

An example is presented:

```
JSON
{
  "to": "user@example.com",
  "subject": "A spot opened for you!",
  "type": "WAITLIST_PROMOTION",
  "params": {
    "name": "John Doe",
```

```
    "eventName": "Tech Summit",
    "eventDate": "2025-06-01",
    "location": "Conference Room B"
  }
}
```

f. CAPACITY_REACHED

Param	Type	Description
`name`	string	Admin or responsible user's name
`eventName`	string	Event name
`capacity`	number	Maximum attendee capacity reached

An example is presented:

```JSON
{
  "to": "admin@example.com",
  "subject": "Event capacity reached",
  "type": "CAPACITY_REACHED",
  "params": {
    "name": "Admin",
    "eventName": "Tech Summit",
    "capacity": 150
  }
}
```